29 - Test practic APD

Să se implementeze, **folosind C/C++ și MPI**, un algoritm distribuit pentru determinarea numărului de valori prime dintr-un vector cu N elemente, precum și a sumei pozițiilor la care numerele prime se găsesc în vector, folosind P task-uri. Rezolvarea se va face maximizând concurența task-urilor. Rezultatul se va printa de către un task ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către task-ul 0.

Atenție! Task-ul 0 nu are voie să afișeze nimic. Nerespectarea acestei cerințe va duce la un punctaj de **0** pe problemă.

În urma compilării, va rezulta un executabil numit *testapd*, care se va rula astfel: **`mpirun -np P ./testapd N n1 n2 ... nN`** (n1…nN sunt cele N numere din vector). Programul va trebui să afișeze numărul de elemente prime din vector, apoi suma pozițiilor acestora, cu spațiu între ele. Aveți mai jos trei exemple de rulare:

```
$ mpirun -np 3 ./testapd 10 2 6 3 13 21 11 14 1 17 8
5 18

$ mpirun -np 4 ./testapd 3 13 14 15
1 0

$ mpirun -np 2 ./testapd 5 29 32 11 4 18
2 2
```

În primul exemplu de mai sus, vectorul este [2,6,3,13,21,11,14,1,17,8], deci elementele prime se află la pozițiile 0, 2, 3, 5 și 8. În acest caz, numărul de elemente prime este 5, iar suma pozițiilor acestora este 18.

Atenție! Programul submis nu trebuie să printeze altceva decât numărul de elemente prime și suma pozițiilor acestora, în această ordine și în formatul de mai sus. Afișarea se va face din orice task în afară de task-ul 0.

Atenție! Doar task-ul 0 are voie sa citească parametrii programului, deci doar el va cunoaște inițial elementele din vector și numărul lor.

Atenție! Nerespectarea oricărei restricții de la punctele (de Atenție!) anterioare va face ca soluția să fie notată cu 0 (zero).

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru *N*, *P* și *ni* sunt: $0 < N <= 100$, $0 < P <= 8$, $0 <= ni < 100$. Programul trebuie să funcționeze corect pentru orice combinație de *N*, *P* și *ni* din aceste intervale.

Trebuie încărcată pe Moodle o arhivă care să conțină **în rădăcină** sursele, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătură cu implementarea realizată. Indiferent dacă aveți sau nu fișier README, este important să adăugați comentarii în cod.